Übungsblatt 10

AKI/NES Programmierung 1, Wintersemester 2024/2025

Deadline: 15.1.2025, 23:00 Uhr

9.1.2025

Gehen Sie beim Lösen aller Programmieraufgaben strikt nach unserem Schema vor:

1. Schreiben Sie zuerst eine Kurzbeschreibung als Kommentar.
2. Führen Sie dann eine Datenanalyse durch. Schreiben Sie das Ergebnis Ihrer Datenanalyse ebenfalls als Kommentar in die Zeile nach der Kurzbeschreibung.
3. Schreiben Sie dann den Funktionskopf inkl. der Typen für Argumente und Ergebnis hin.
4. Schreiben Sie dann Tests mittels `check`.
5. Setzen Sie die passenden Schablonen in den Rumpf der Funktion ein.
6. Schreiben Sie erst dann den Funktionsrumpf hin.

Eine Lösung, in der nicht ersichtlich ist, dass Sie gemäß des Schemas aggiert haben, erhält *keine* Punkte. Insbesondere erhält eine Lösung, bei der nur der Funktionsrumpf da steht, keine Punkte, auch wenn die Funktion korrekt ist.

Dahingegen erhalten Sie Teilpunkte wenn Sie dem Schema folgen, aber an einem gewissen Punkt nicht weiterkommen.

Aufgabe 1 (5+5+5 Punkte)

Schreiben Sie ein Programm, dass in einer Textdatei die am häufigsten vorkommenden Wörter findet. Gehen Sie wie folgt vor:

(a) Implementieren Sie eine Funktion `stringToWords`, die für einen String eine Liste aller darin enthaltener Wörter zurückgibt. Dabei sollen Satzzeichen am Ende von Wörtern entfernt werden.

Beispiel:

```
Hallo Stefan,

wie geht es dir?
```

Für diesen Strings sollte die Liste `["Hallo", "Stefan", "wie", "geht", "es", "dir"]` zurückgegeben werden.

(b) Implementieren Sie eine Funktion `countWords`, die für eine Liste von Strings ein Dictionary berechnet, so dass für jedes Wort aus der Liste ein Eintrag im Dictionary steht, der angibt, wie häufig das Wort vorkommt. Groß- und Kleinschreibung soll dabei keine Rolle spielen.

Beispiel: für die Liste `["Ding","einmal","ding"]` soll `{"ding":2,"einmal":1}` zurückgegeben werden.

(c) Implementieren Sie jetzt die Funktion `countWordsInFile`. Diese Funktion bekommt den Namen einer Datei übergeben und gibt die drei am häufigsten in der Datei vorkommenden Wörter auf der Konsole mit ihrer Häufigkeit sortiert ausgeben. Beispielausgabe:

```
er: 31
haus: 29
bunt: 19
```

Hinweis: Sie müssen Tests für Teile (a) und (b) schreiben. Testen Sie Teil (c) interaktiv. In Moodle steht dazu eine Testdatei `hitchhikers.txt` bereit. Die Ausgabe für diese Datei sollte so aussehen:

```
the: 10
was: 9
he: 7
```

Hinweis: Sie können ein Dictionary `d` wie folgt sortieren und erhalten dann eine Listen von Paaren, wobei vorne in der Liste die am häufigsten vorkommenden Wörter stehen:

```
sorted(d.items(), key=lambda x: x[1], reverse=True)
```

Speichern Sie ihre Lösung in einer Datei `wordcount.py`

Aufgabe 2 (2+3+5+5+5 Punkte)

Entwerfen und implementieren Sie einen Record `Fraction` zum exakten Rechnen mit Brüchen auf der Basis der vorhandenen arithmetischen Operationen auf `int`. Dabei hat ein Bruch zwei Eigenschaften: `numerator` (Zähler) und `denominator` (Nenner).

Stellen Sie in Ihrer Implementierung sicher, dass folgende Eigenschaften gelten:

- Der Nenner darf nie 0 sein.
- Instanzen von Fraction sollen nicht veränderbar sein.
- Die Implementierung soll komplett ohne Gleitkommazahlen auskommen!

Sie dürfen ausschließlich folgenden Imports nutzen:

```
from wypp import *
from math import gcd
```

Insbesondere dürfen Sie das Module `fractions` nicht verwenden!

Gehen Sie wie folgt vor:

(a) Verwenden Sie Records zur Repräsentation von Brüchen

(b) Stellen Sie zunächst die Bedingung „Nenner ungleich 0“ sicher. Dazu bieten Records die Möglichkeit, durch Defininition einer Method

```
__post_init__(self) -> None
```

Code direkt nach Konstruktion eines Wertes auszuführen. Werfen Sie einen `ValueError` wenn die Bedingung verletzt ist.

Schreiben Sie einen Test, der sicherstellt, dass der Fehler auch wirklich auftritt. Dazu sollen Sie `try-except` verwenden. Wenn kein Fehler oder eine falscher Fehlerart auftritt, können Sie mit `checkFail('...')` das Fehlschlagen des Tests signalisieren. (Ersetzen Sie `'...'` durch eine sinnvolle Fehlermeldung.)

(c) Schreiben Sie eine Methode `cancel(self: Fraction) -> Fraction`, die einen Bruch vollständig kürzt und normalisiert. So soll z.B. der Bruch $\frac{2}{-6}$ zu $\frac{-1}{3}$ werden.

(d) Implementieren Sie die Operatoren `+`, `-` und `*` auf Brüchen. Geben Sie nur vollständig gekürzte Brüche als Ergebnis zurück. Welche Methoden Sie dazu implementieren müssen, finden Sie im Abschnitt 3.3.8 unter `https://docs.python.org/3/reference/datamodel.html`

(e) Implementieren Sie Methoden für die Vergleichsoperatoren `==` und `<` auf Brüchen. Sie finden die hierfür passenden Methoden im Abschnitt 3.3.1 unter `https://docs.python.org/3/reference/datamodel.html`

Speichern Sie ihre Lösung in einer Datei `fraction.py`

Gesamtpunkte: 35

Abgabe

Via Upload im Praktomat bis **15.1.2025, 23:00 Uhr**. Speichern Sie jede Programmierenaufgabe in eine eigene Datei. Der Name der Datei muss dabei `aufgabe_0`N`.py` sein, wobei N die Nummer der Aufgabe ist. Zum Beispiel lautet für Aufgabe 1 der Name der Datei dann `aufgabe_01.py`.

Verpacken Sie alle Dateien in einem zip-Archiv (Standard-Zip, keine 7zip, rar o.ä.), welches den Namen `NAME_MATRIKELNUMMER.zip` hat. Ersetzen Sie dabei `NAME` und `MATRIKELNUMMER` durch Ihren Namen und Ihre Matrikelnummer. Es ist wichtig, dass Sie sich an die Namenskonventionen halten, die wir Abgaben automatisch überprüfen.

Wichtig: Die Lösungen für **alle** Programmieraufgaben müssen ohne Fehler ausführbar sein, sonst akzeptiert der Praktomat Ihre Abgabe nicht. Wenn der Praktomat nach dem Upload für eine oder mehrere Aufgaben den Status **failed** anzeigt, wird ihre **komplette Abgabe nicht akzpetiert** und Sie erhalten für das **komplette Blatt 0 Punkte!** Kommentieren Sie fehlerhafte Codestellen in einem solche Fall aus, so dass der Praktomat Ihre Abgabe akzeptiert.